SLOVENSKÁ TECHNICKÁ UNIVERZITA V BRATISLAVE

Fakulta elektrotechniky a informatiky

Autonómna misia dronu v ROS 2

LRS – Zadanie 1

2025/2026

Bc. Šimon Maco, Bc. Boris Šupák

Obsah

1 Analýza zvolených prístupov **2**
1.1 Reprezentácia mapy . . . 3
1.2 Nafúknutie prekážok . . . 3
1.3 Plánovací algoritmus . . . 4
1.4 Úprava cesty . . . 5
1.5 Polohový regulátor . . . 6
1.6 Riadenie natočenia . . . 7
1.7 Geodetická konverzia a výška . . . 7
1.8 Konvencie uhlov . . . 8
1.9 Parametre osmičky a geofence . . . 8

2 Diagramy riešenia **9**
2.1 Cesta spracovania dát . . . 9
2.2 Riadiaci diagram ROS . . . 10
2.3 Súbežnosť – kto berie zámok . . . 11

3 Namerané výsledky **12**
3.1 Súhrn . . . 12
3.2 Indoor misia . . . 13
3.3 Outdoor misia . . . 14

4 Známe obmedzenia **16**

1 Analýza zvolených prístupov

Pri každej téme je uvedené, čo sme zvolili a ako to funguje, prečo práve takto, a alternatívy s výhodami a nevýhodami. Čísla sú z rosbagov `bags/indoor_run1` a `bags/outdoor_run1`. Tabuľka 1 je prehľad všetkých rozhodnutí. Celý uzol je v jednom súbore `src/lrs_mission_node.cpp` balíka `lrs_mission`; pri každej téme je uvedené, v ktorých funkciách sa nachádza.

Tabuľka 1: Prehľad rozhodnutí

Téma	Voľba	Hlavný dôvod
1.1 Mapa	3D voxelová mriežka 0,1 m	dopyt $O(1)$, celá hala má len 3,2 MB
1.2 Nafúknutie	guľa 0,5 m, regály ako kvádre	polovica dronu + tolerancia hard + rezerva
1.3 Plánovanie	A*, 26 susedov	optimálna cesta na mriežke za 3–13 ms
1.4 Úprava cesty	skracovanie cez priamku	45–114 → 2–9 bodov, každý úsek overený
1.5 Regulátor	GUIDED + carrot setpoint	dron ide po čiare plánu, nereže rohy
1.6 Natočenie	nos v smere letu / na POI / tangenta	pravidlo 4 zadania, bez skokov o 360°
1.7 Geodézia	lokálna rovina okolo home	chyba pod 1 mm na ploche misie
1.8 Uhly	všade ENU v radiánoch	jedna konvencia, prevody len na okrajoch
1.9 Osmička	$A = 3$ m, $\omega = 0{,}3$ rad/s	max. 1,27 m/s, pod limitom 1,5 m/s

1.1 Reprezentácia mapy

Voľba: hustá 3D voxelová mriežka (occupancy grid) s rozlíšením 0,1 m.
Ako to funguje: PCL načíta `map.pcd` (620 311 bodov) a `pcl::VoxelGrid` ho zredukuje na jeden bod na voxel (146 524 bodov). Z nich sa vyplní pole $183 \times 144 \times 60$ voxelov, kde každá bunka je obsadená alebo voľná. Otázka „je bod voľný?“ je len prepočet súradníc na index poľa. Všetko mimo mriežky sa považuje za obsadené.
V kóde: trieda `VoxelMap`; mapu načíta `setupIndoor()` cez `pcl::io::loadPCDFile` a `pcl::VoxelGrid`, mriežku vyplní `VoxelMap::build()`. Dopyt na bod robia `toIndex()` a `free()`.

- **Voxel grid, nie octree** – hala je malá: mriežka má 1,58 M voxelov, 1 bajt na voxel. Uzol drží dve kópie (`raw` a `inflated`, pozri 1.2), spolu 3,2 MB.
- Pri štarte uzol vypíše bounding box mapy a počet voxelov.
- **Prečo 0,1 m:** voxel musí byť menší ako tolerancia hard (0,15 m); chyba zaokrúhlenia je najviac polovica uhlopriečky voxelu $\approx 0{,}09$ m; dron (0,35 m) má 3–4 voxely a nafúknutie 0,5 m = 5 voxelov, takže guľa sa aproximuje dobre.
- 0,05 m by znamenalo 8× viac voxelov (12,6 M) a pomalší A*; 0,25 m by zaokrúhľovalo až o 0,2 m, viac ako tolerancia hard.

Alternatívy	Výhody	Nevýhody
– OctoMap (octree) – k-d strom nad mračnom – 2D mapa / vrstvy (zadanie ich nepovoľuje)	– dopyt $O(1)$ – A* beží priamo na mriežke – plné 3D – prelet ponad regál	– pamäť rastie s objemom → pre veľké priestory nevhodné – regály treba doplniť ručne (1.2)

1.2 Nafúknutie prekážok

Voľba: guľa s polomerom 0,5 m okolo každého obsadeného voxelu, regály ako plné kvádre.
Ako to funguje: najprv sa do mriežky „vypália“ regály ako plné kvádre, potom sa okolo každého obsadeného voxelu označí guľa s polomerom `inflation_radius`. Plánovač potom hľadá cestu pre bod – stred dronu – a nafúknutie zaručí, že telo dronu má od prekážky odstup.
V kóde: `VoxelMap::fillBox()` vyplní každý kváder z parametra `rack_boxes`, potom `VoxelMap::inflate()` prejde obsadené voxely a označí okolo nich guľu. Obe volá `setupIndoor()`.

- **0,5 m = 0,25 m** polovica uhlopriečky dronu **+ 0,15 m** tolerancia hard **+ 0,10 m** rezerva. Rozmer dronu $0{,}347 \times 0{,}347$ m je kolízny box z `fei_lrs_gazebo.world`.
- Nafukuje sa v 3D (guľa) – rovnaký odstup zboku aj zhora; overené: prelet 0,56 m nad regálom.
- **Regály:** mračno zachytí len povrchy políc a medzi poschodiami sú voľné voxely. Každý rad regálov je preto vyplnený ako plný kváder (`rack_boxes` v `indoor.yaml`) = obal 116 kolíznych boxov z `models/fei_lrs_racks/model.sdf`, od podlahy do 4,10 m.
- 190 397 obsadených voxelov → 836 630 po nafúknutí; polomer je parameter `inflation_radius`.

Alternatívy	Výhody	Nevýhody
– kolízna kontrola tvarom dronu – distance field (ESDF) s cenou – iné nafúknutie vo zvislom smere	– plánovač rieši len bod – jednoduché a rovnaké v x, y aj z – regály nemôžu prepustiť cestu	– konzervatívne – úzke priechody sa zatvoria – zmenšuje priestor aj pri podlahe a strope – bod misie v zóne: A* plánuje len k najbližšiemu voľnému voxelu

1.3 Plánovací algoritmus

Voľba: A* na 26-susednej mriežke, euklidovská heuristika.
Ako to funguje: A* rozširuje voxely v poradí podľa $f = g + h$, kde g je doterajšia dĺžka cesty a h odhad zvyšku. Z každého voxelu sa dá ísť do 26 susedov (aj diagonálne a šikmo hore/dole), cena kroku je jeho euklidovská dĺžka. Plánuje sa zvlášť každý úsek medzi dvoma riadkami misie.
V kóde: funkcia `astar()` s prioritnou frontou (`std::priority_queue`); štart a cieľ v nafúknutej zóne posunie `nearestFree()`. Volá ju `setupIndoor()` pre každý úsek misie.

- **Heuristika:** euklidovská vzdialenosť do cieľa – nikdy nepreceňuje (priamka je najkratšia), je prípustná → cesta je **optimálna na mriežke**.
- **Čas:** 3–13 ms na úsek v nahrávke, celá misia pod 40 ms; príprava mapy 590 ms.
- **Cesta neexistuje:** otvorený zoznam sa vyprázdni a plánovač vráti chybu, nezamrzne.
- **Bod misie v nafúknutej zóne:** A* plánuje k najbližšiemu voľnému voxelu do 1 m (`nearestFree()`). Posledný setpoint úseku je však vždy presný bod misie, takže tento krátky kúsok sa voči nafúknutiu nekontroluje.

Alternatívy	Výhody	Nevýhody
– Dijkstra (bez heuristiky, pomalší) – RRT / RRT* (náhodné, neoptimálne) – Theta* (any-angle)	– optimálne a deterministické – milisekundy na úsek – spoľahlivo zistí, že cesta neexistuje	– schodovitá cesta → treba úpravu (1.4) – optimálne len na mriežke, nie v spojitom priestore – čas a pamäť rastú s objemom

1.4 Úprava cesty

Voľba: skracovanie cez priamu viditeľnosť (line-of-sight shortcutting).
Ako to funguje: od každého ponechaného bodu sa hľadá najvzdialenejší ďalší bod, ku ktorému vedie priamka bez kolízie. Priamka sa kontroluje každú polovicu voxelu (5 cm) voči nafúknutej mriežke. Body medzi nimi sa zahodia.
V kóde: funkcia `simplify()` a priamku kontroluje `VoxelMap::segmentFree()`. Po zjednodušení `setupIndoor()` znova overí každý úsek a vypíše „collision check OK“.

- **Odstránené:** všetky body, ktoré sa dajú preletieť priamkou – nielen kolineárne.
- **Bezkolíznosť:** každý výsledný úsek sa znova overí; log „collision check OK“ pri všetkých úsekoch.
- **Body pred a po** pre všetkých 6 úsekov misie `exam_example.csv` sú v tabuľke 2.

Alternatívy	**Výhody**	**Nevýhody**
– B-spline vyhladenie – any-angle plánovač (Theta*) – len odstránenie kolineárnych bodov	– 45–114 → 2–9 bodov – každý úsek znova overený – jednoduchá implementácia	– ostré rohy, nie hladká trajektória – greedy – nemusí byť najkratšia – úsek môže viesť tesne pri hranici nafúknutia

Tabuľka 2: Úseky misie `exam_example.csv`

Úsek	A* čas	Body pred → po	Kontrola kolízií
1 → 2	5 ms	50 → 2	OK
2 → 3	3 ms	45 → 4	OK
3 → 4 (ponad regál)	13 ms	69 → 9	OK
4 → 5	7 ms	69 → 3	OK
5 → 6	3 ms	61 → 4	OK
6 → 7	8 ms	114 → 4	OK

1.5 Polohový regulátor

Voľba: regulátor ArduPilota v režime GUIDED + vlastný „carrot“ setpoint a tri úrovne presnosti.
Ako to funguje: uzol posiela 20× za sekundu polohu a yaw na `/mavros/setpoint_raw/local`. Setpoint neskočí hneď do cieľa, ale posúva sa po úseku rýchlosťou `max_speed` (1 m/s indoor, 1,5 m/s outdoor), takže dron ide po naplánovanej čiare a nereže rohy. Samotné riadenie motorov robí ArduPilot.
V kóde: funkcia `go()` – carrot setpoint a podmienky pass/soft/hard; setpoint posiela `send()` na `/mavros/setpoint_raw/local`. Volá ich `tick()` 20× za sekundu. Tolerancie sú parametre `pass/soft/hard_tolerance` v `indoor.yaml`.

- **pass** (body cesty od plánovača): 0,5 m, bez zastavenia – ďalší setpoint ide hneď, ako je dron bližšie.
- **soft**: 0,4 m, bez zastavenia; **hard**: 0,15 m **a** rýchlosť $< 0{,}15$ m/s, splnené nepretržite 1 s (pri porušení sa časovač nuluje).
- **Prečo:** 0,15 m je v odporúčanom rozsahu 10–20 cm a je zahrnutá v nafúknutí; soft je prísnejšia než pass, aby boli body misie presnejšie ako body cesty; pass ≈ 5 voxelov.
- **Namerané na hard:** 0,06–0,08 m (wp 3, 4, 5, 7), outdoor 0,03 a 0,10 m.
- **Nezastavuje na každom bode:** presnosť majú len riadky misie, body cesty sú pass; po úprave cesty ich je len 2–9 na úsek.
- Yaw úloha: $|\text{chyba}| < 5°$, držané 1 s.

Alternatívy	**Výhody**	**Nevýhody**
– pevný vzdialený setpoint – otestované: ∼3 m/s a len 0,23 m od regálu – rýchlostné setpointy s vlastným PID	– obmedzená rýchlosť – dron drží čiaru plánu – odladený regulátor ArduPilota	– dron zaostáva za setpointom (medián 0,60 / 0,84 m) – bez profilu zrýchlenia a brzdenia – závisí od ladenia ArduPilota

1.6 Riadenie natočenia

Voľba: nos v smere úseku; na štvorci na POI; na osmičke tangenta; vždy s unwrapom.
Ako to funguje: pred začiatkom úseku sa vypočíta smer atan2($\Delta y, \Delta x$). Ak je otočka väčšia ako 30°, dron sa najprv otočí na mieste, kým chyba neklesne pod 10°, a až potom letí. Na štvorci sa každý tick počíta azimut na POI, na osmičke analytická tangenta. Pri oboch sa posiela aj derivácia (yaw rate) ako feed-forward, aby ArduPilot nezaostával.
V kóde: smer úseku počíta `start()` (`travel_yaw_`), otočku na mieste a azimut na POI rieši `go()`, tangentu osmičky `figure8()`. Unwrap je v `send()`, funkcia `wrap()` drží uhol v $(-\pi, \pi]$.

- **Unwrap:** nový yaw = predošlý + wrap(rozdiel) → setpoint nikdy neskočí o 360°.
- **POI:** mieri sa len yaw (horizontálny azimut), gimbal ani pitch sa neriadi.
- **Yaw rate:** analytický – derivácia azimutu z rýchlosti, pri osmičke $\omega \cdot (\dot{x}\ddot{y} - \dot{y}\ddot{x})/(\dot{x}^2 + \dot{y}^2)$.
- **Namerané:** nos vs. smer letu indoor 0,7°, nos vs. POI 3,1°, yaw vs. príkaz počas celého outdoor letu 1,0°.

Alternatívy	Výhody	Nevýhody
– plynulé natáčanie počas letu – auto-yaw ArduPilota	– spĺňa pravidlo 4 zadania – feed-forward → malá chyba na POI – žiadne skoky o 360°	– otočka na mieste predlžuje misiu – tangenta v setpointe, nie v polohe dronu → na osmičke nos ∼24° od smeru letu

1.7 Geodetická konverzia a výška

Voľba: lokálna dotyková rovina (flat Earth) okolo home, $R = 6\,378\,137$ m.
Ako to funguje: pred vzletom uzol čaká na GPS fix, min. 6 satelitov a home pozíciu. POI a rohy štvorca sa zadajú ako lat/lon (alebo posun od home) a prepočítajú sa na metre: východ = $\Delta\text{lon} \cdot R \cdot \cos(\text{lat}_0)$, sever = $\Delta\text{lat} \cdot R$. Opačný smer sa používa na výpis, kde dron práve je.
V kóde: štruktúra `Geo` – `toEnu()` (lat/lon → metre) a `toGeo()` (opačne); čakanie na GPS fix je v `gpsReady()`, POI a rohy štvorca prepočíta `buildOutdoorActions()`.

- **Chyba:** rastie približne ako d^2/R – pri ∼8 m od home je pod 1 mm.
- **Výška:** príkaz aj kontrola v lokálnom $z - z_{\text{home}}$ (5 m nad štartom); `rel_alt` sa len vypisuje na porovnanie, AMSL sa nepoužíva.
- Žiadna absolútna súradnica v kóde – všetko je relatívne k home z danej relácie.

Alternatívy	Výhody	Nevýhody
– WGS-84 → ECEF → ENU (GeographicLib) – UTM	– jednoduché, bez ďalšej knižnice – chyba zanedbateľná na tejto ploche	– nepresné na desiatky km – ignoruje sploštenie Zeme – presnosť závisí od home pozície

1.8 Konvencie uhlov

Voľba: v kóde všade ENU v radiánoch; prevody len na vstupe a výstupe.
Ako to funguje: každý uhol sa pri vstupe prevedie do jednej konvencie – ENU (0 = východ, proti smeru hodín) v radiánoch – a v nej sa počíta všetko. Stupne sú len v CSV, parametroch a logoch.
V kóde: `toLocal()` otočí bod z mapy o `map_to_local_yaw_deg` (90°) do lokálneho ENU; `setupIndoor()` pri čítaní `yawN` pripočíta tú istú rotáciu. Prevod kompasu na ENU je v `printStatus()`.

- **yawN v misii:** N stupňov **absolútne** v rámci mapy (nie relatívne k aktuálnemu smeru), proti smeru hodín od osi $+X$ mapy; do lokálneho ENU sa pripočíta 90°.
- **Mapa → lokál:** local = $R_z(90°) \cdot$ map – namerané: dron na mape (13, 7) hlási lokálne (−7, 13).
- **NED:** náš kód v NED nepočíta nikde – MAVROS prevádza ENU setpointy do NED pre ArduPilot.
- **Kompas:** 0 = sever, v smere hodín → $\text{yaw}_{\text{ENU}} = 90° - \text{heading}$; overené za behu, uzol každú sekundu vypisuje kompas aj yaw z pózy.

Alternatívy	Výhody	Nevýhody
– počítať v NED ako ArduPilot	– jedna konvencia v celom kóde – zhoda s MAVROS local ENU – kontrola súradníc v bagu: 0,000 m	– viac prevodov na vstupe = miesta pre chybu znamienka – rotácia 90° platí pre tento svet

1.9 Parametre osmičky a geofence

Voľba: $A = 3$ m, $\omega = 0{,}3$ rad/s, 1 kolo; $x = A \sin t$, $y = A \sin t \cos t$, $t = \omega \cdot (\text{teraz} - t_0)$.
Ako to funguje: timer s frekvenciou 20 Hz v každom ticku vypočíta bod krivky aj tangentu a pošle ich ako setpoint. Pred štartom dron preletí do stredu (POI) a otočí sa na tangentu v $t = 0$ (45°), aby nebol prudký skok.
V kóde: funkcia `figure8()` – bod krivky, tangenta a yaw rate v každom ticku; poradie akcií (stred, natočenie na 45°, osmička, návrat) zostaví `buildOutdoorActions()`. Parametre `figure8_amplitude`, `figure8_omega` a `figure8_laps` sú v `outdoor.yaml`.

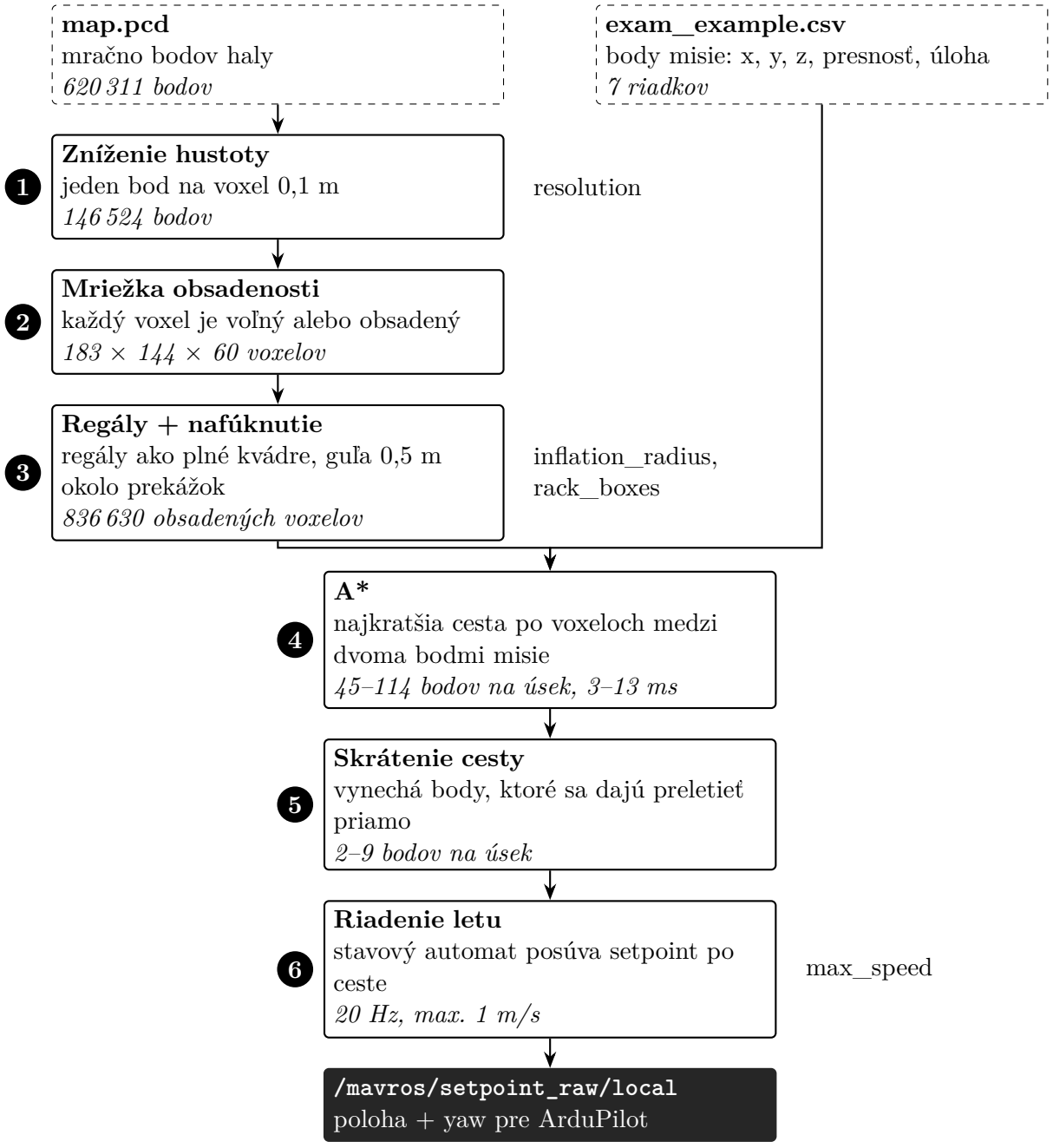
- **Max. rýchlosť:** $A\omega\sqrt{2} = 3 \cdot 0{,}3 \cdot 1{,}41 = 1{,}27$ m/s $< 1{,}5$ m/s; kolo $2\pi/\omega = 21$ s (namerané 20,9 s).
- **Geofence:** najďalej od štartu je roh štvorca 7,9 m (osmička 6,9 m), výška 5 m; misia sa zmestí do obdĺžnika $6 \times 7{,}5$ m od štartu.
- ω, A, POI aj výška sú parametre v `outdoor.yaml`.

Alternatívy	Výhody	Nevýhody
– vyššia ω (rýchlejšie) – viac kôl	– rezerva pod limitom rýchlosti – odchýlka max 0,041 m – yaw aj yaw rate analyticky	– pomalé – vyššia ω → väčšie oneskorenie a chyba natočenia – na reálnom drone (vietor) treba overiť

2 Diagramy riešenia

2.1 Cesta spracovania dát

Od mapy a súboru misie až po setpointy posielané do MAVROS. Kroky 1–3 pripravia mapu prekážok raz pri štarte, krok 4 v nej hľadá cestu medzi každými dvoma bodmi misie, krok 5 cestu skráti a krok 6 ju počas letu posiela dronu.



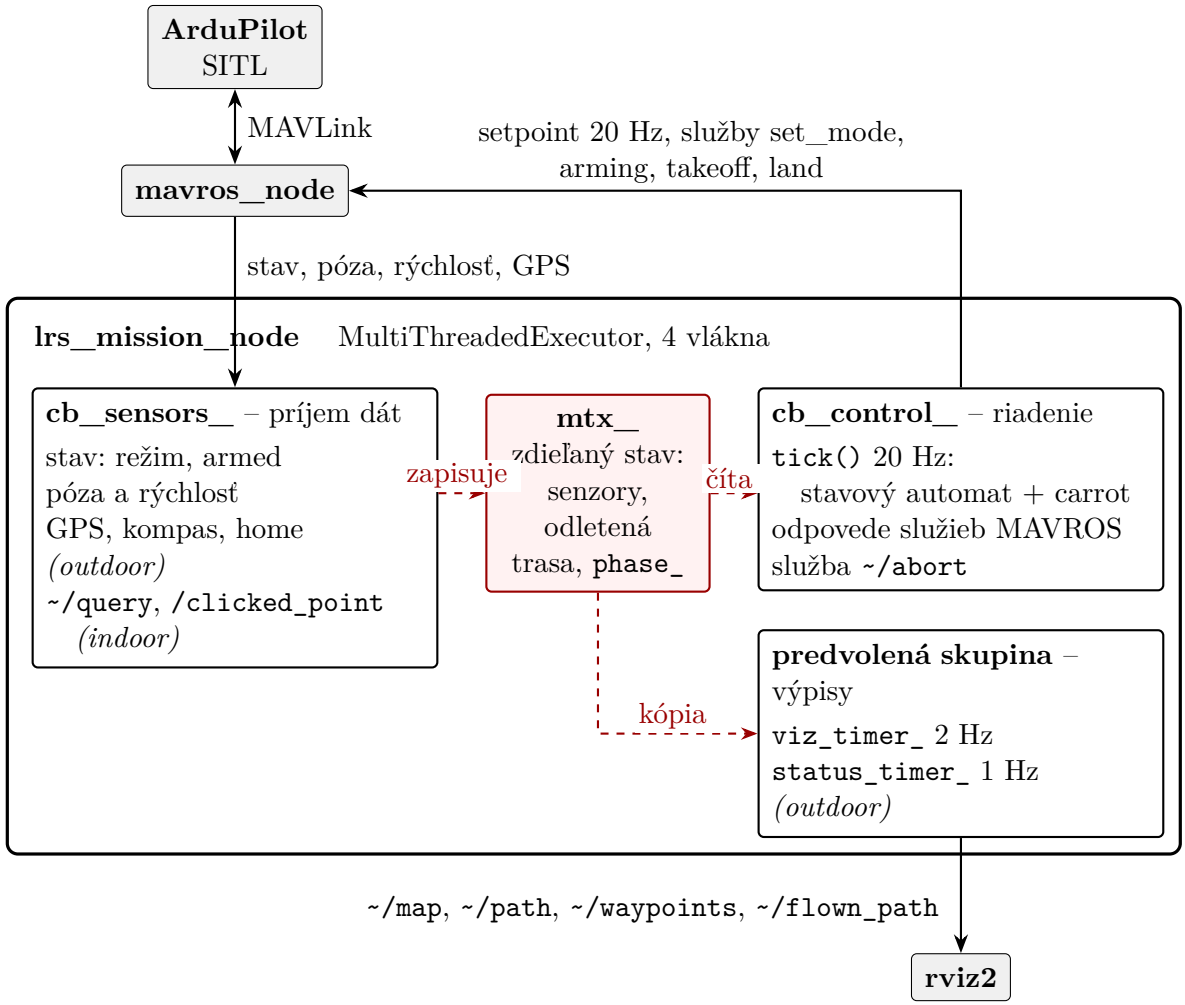
Obr. 1: Indoor spracovanie dát v šiestich krokoch. Vpravo od kroku je parameter z `config/indoor.yaml`, ktorý ho ovplyvňuje; čísla sú namerané na `exam_example.csv`.

Vonku (outdoor) kroky 1–5 nahrádza `buildOutdoorActions()`: nie je tu mapa ani plánovač. Rohy štvorca a POI sú zadané geodeticky a prepočítajú sa na lokálne ENU metre

okolo domovskej pozície. Osmička sa v každom ticku počíta z parametrických rovníc, nie je to zoznam waypointov.

2.2 Riadiaci diagram ROS

Uzly, topicy a služby a to, v ktorej **callback group** beží každý callback. Všetky skupiny sú `MutuallyExclusive` a bežia súbežne na `MultiThreadedExecutor` so 4 vláknami. Plné šípky sú ROS topicy a služby, prerušované šípky prístup k zdieľanému stavu pod zámkom `mtx_`.



Obr. 2: ROS graf uzla `lrs_mission_node`: tri callback groups, zdieľaný stav pod mutexom `mtx_` a spätná väzba do MAVROS

2.3 Súbežnosť – kto berie zámok

Tabuľka 3: Callbacky, ich skupiny a práca so zámkom `mtx_`

Callback	Skupina	Berie `mtx_`?	Dôvod
`tick()` 20 Hz	`cb_control_`	áno, celý tick	číta polohu, rýchlosť a stav, ktoré zapisuje `cb_sensors_`. Tick je krátky (výpočet + publikovanie setpointu), senzory čakajú len zlomok ms
odpovede služieb, `~/abort`	`cb_control_`	len `~/abort`	odpovede menia len `step_` a `pending_`, ktoré iná skupina nečíta – s `tick()` ich serializuje spoločná skupina. `~/abort` zapisuje `phase_`, ktoré číta `recordFlown()` z `cb_sensors_`
odbery pose / state / GPS	`cb_sensors_`	áno, zapisuje	snapshot senzorov a odletenú trasu čítajú iné skupiny
`viz_timer_` 2 Hz	predvolená	len kópia	pod zámkom skopíruje `flown_`, Path (1000+ póz) serializuje až po jeho uvoľnení
`printStatus()` 1 Hz	predvolená	len kópia	skopíruje svojich osem hodnôt, vypisuje mimo zámku

Výsledok: 20 Hz riadiaci timer nikdy nečaká na pomalú vizualizáciu – pomalá práca (serializácia trasy, formátovanie výpisu) beží na kópii mimo zámku. Premenné stavového automatu (`idx_`, `step_`, `pending_`) zámok nepotrebujú, lebo ich mení len skupina `cb_control_`.

3 Namerané výsledky

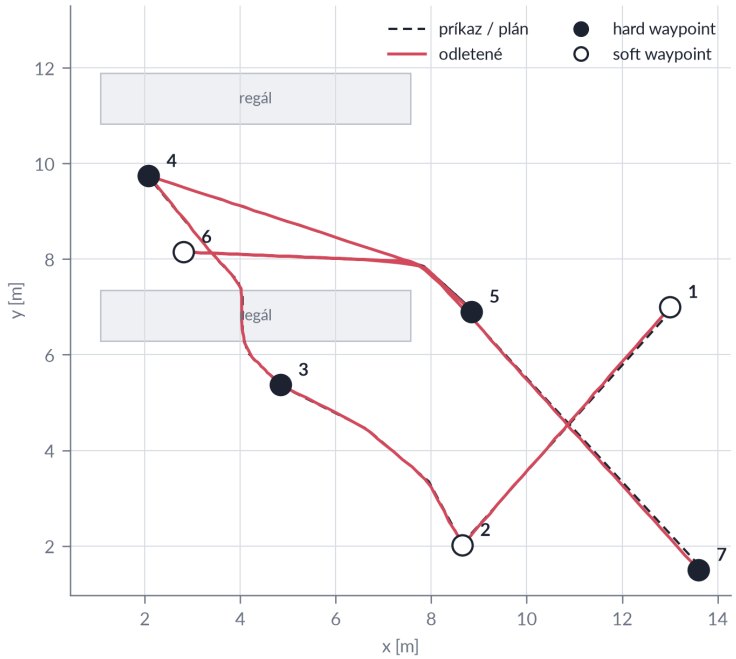
- Zdroj: rosbagy `indoor_run1` (191 s) a `outdoor_run1` (103 s), nahraté 13. 10. 2026, spracované cez `lrs_mission/scripts/plot_bag.py`.
- Uhlové chyby: medián a p95 – maximum skresľuje skoková odozva pri otočke.
- V grafoch: **čierna prerušovaná** = príkaz, **červená** = odletené.

3.1 Súhrn

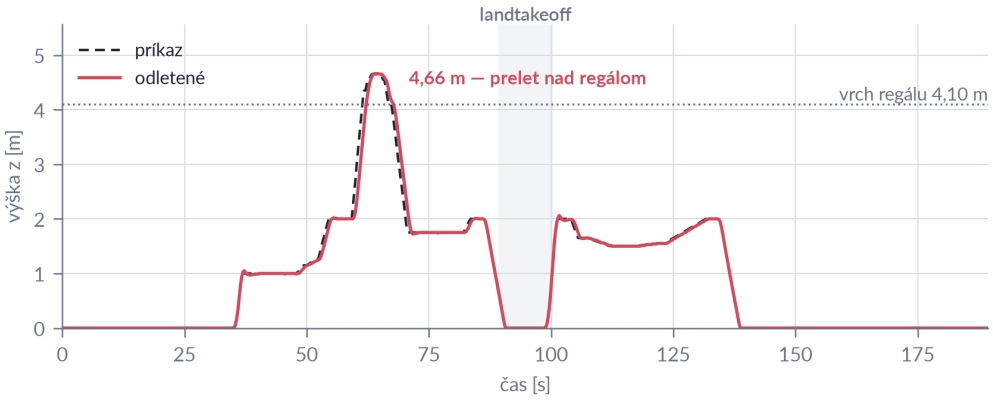
Tabuľka 4: Súhrn nameraných metrík

Metrika	Indoor	Outdoor	Čo to dokazuje
Trvanie	191 s	103 s (misia 78,7 s)	outdoor: IDLE → … → DONE bez prerušenia
Kontrola súradníc	0,000 m	0,000 m	transformácia mapa ↔ lokál sedí
Yaw vs. príkaz	medián 0,6°	medián 1,0°, p95 5,9°	dron drží príkazový smer
Nos vs. smer letu	medián 0,7°	medián 24° (osmička)	pravidlo 4; outdoor: dron zaostáva za setpointom
Nos vs. POI	–	medián 3,1°	A1.3 §3: nos na POI
Odchýlka od cesty	max 0,080 m	osmička max 0,041 m	presné sledovanie trasy
Oneskorenie za setpointom	medián 0,60 m	medián 0,84 m	vysvetľuje 24° na osmičke
Max. rýchlosť	vodorovne 1,08 m/s zvislo 1,32 m/s	vodorovne 1,55 m/s zvislo 2,16 m/s	zvislá = vzlet, riadi ho ArduPilot; vodorovná ≈ carrot `max_speed`

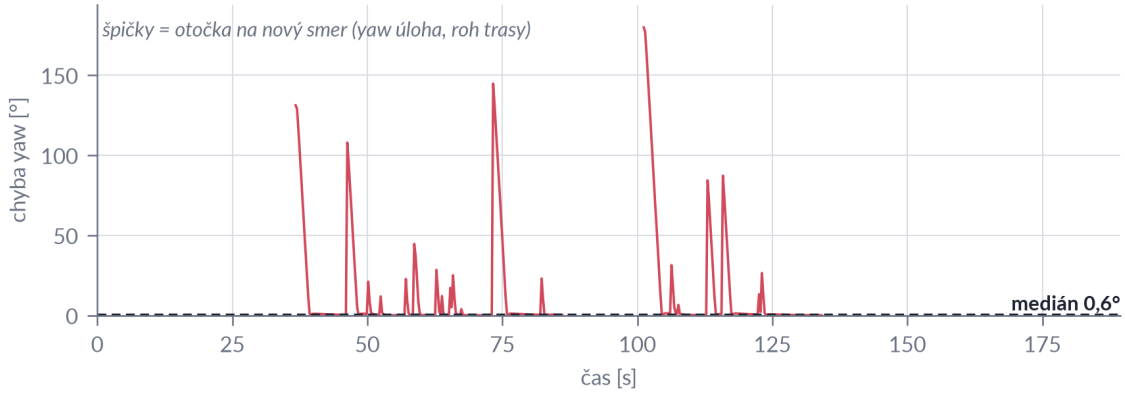
3.2 Indoor misia



Obr. 3: Indoor – pôdorys. Odletená trasa leží na plánovanej (odchýlka max 0,08 m) a obchádza oba regály; plný bod = hard waypoint, prázdny = soft.

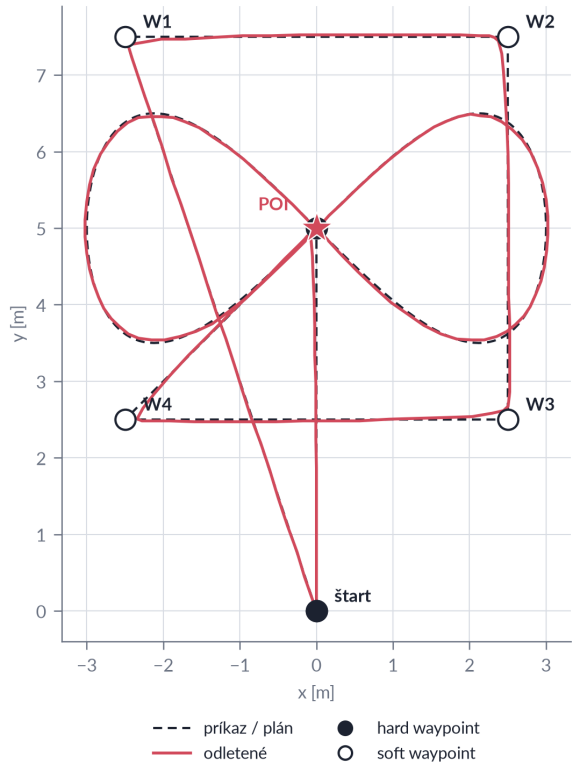


Obr. 4: Indoor – výška. Prelet **4,66 m nad regálom (4,10 m)** → 3D plánovanie (A1.1 §3); rezerva 0,56 m ≈ polomer nafúknutia 0,5 m; šedý pás = landtakeoff na waypointe 5.

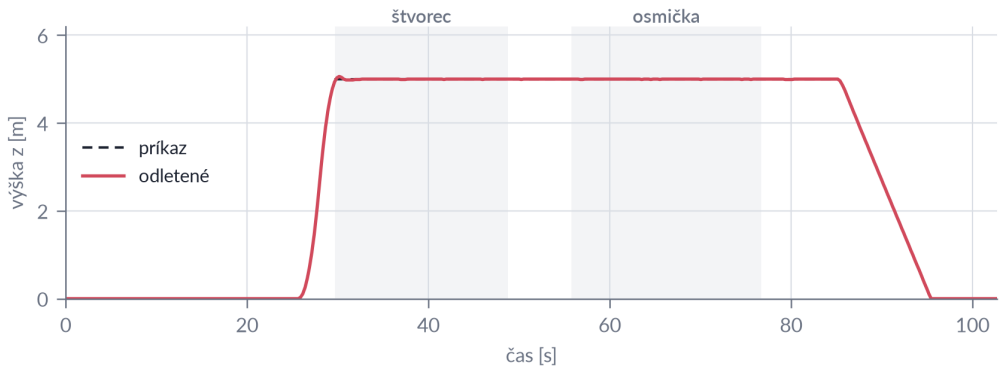


Obr. 5: Indoor – chyba natočenia. Medián 0,6°, mimo otočiek takmer nulová; špičky = otočka na nový smer pri yaw úlohe alebo v rohu.

3.3 Outdoor misia



Obr. 6: Outdoor – pôdorys. Štvorec 5 × 5 m (W1–W4) s nosom stále na POI, osmička 6 × 3 m okolo POI (odchýlka max 0,041 m), návrat na štart.



Obr. 7: Outdoor – výška. Stúpanie na 5 m, výška konštantná počas štvorca aj osmičky; osmička trvala 20,9 s (predpoveď 21 s).



Obr. 8: Outdoor – chyba natočenia. Medián 1,0°, p95 5,9°; jediná špička okolo 77 s = skok príkazového smeru po osmičke.

4 Známe obmedzenia

- **Oneskorenie za setpointom** 0,6–0,84 m: carrot nemá profil zrýchlenia a brzdenia, preto nos na osmičke odchádza od skutočného smeru letu o ∼24°. Setpoint je ∼0,79 m (∼0,72 s) pred dronom a tangenta sa otáča ∼34°/s, takže $0{,}72 \times 34 \approx 24°$ – je to oneskorenie polohového regulátora, nie chyba riadenia yaw.
- **Regály vs. text zadania:** zadanie opisuje kolíziu regálu ako plný kváder $6{,}5 \times 3{,}76 \times 5{,}59$ m. V aktuálnom svete (`models/fei_lrs_racks/model.sdf`) je regál zložený zo 116 kolíznych boxov s vrchom vo výške 4,10 m, a podľa toho sú nastavené `rack_boxes` – úsek $3 \to 4$ preto letí ponad regál vo výške 4,66 m. Ak by sa hodnotilo vo svete s vyšším kvádrom, stačí zmeniť výšku v `rack_boxes` a plánovač pôjde okolo.
- **Len simulácia:** GPS v SITL je takmer bez šumu; s reálnym GPS a kompasom misia nebola odskúšaná.
- **Názvy topicov:** prefix `/mavros` je v kóde pevný; na reálnom drone sú topicy pod `/droneX`, preto by prefix musel byť parameter.
- **Bod misie v nafúknutej zóne:** plán končí v najbližšom voľnom voxeli a posledný kúsok k presnému bodu (najviac 1 m) dron letí bez kontroly voči nafúknutiu. Bod misie tesne pri prekážke by tak mohol viesť dron bližšie k nej, než dovoľuje nafúknutie.
- **Mapa** je hustá mriežka – pre väčší priestor by bola potrebná octree.